

Репликация MySQL: master–slave, кворум и файловер

1. Топология и конфигурация

Параметр	Значение
СУБД	MySQL 8.0 (3 узла в Docker Compose)
Топология	1 master (:3406) + 2 slave (:3407, :3408)
Тип репликации	Потоковая, GTID-based, ROW binlog
Кворум	Полусинхронная (semi-sync), ACK от ≥1 из 2 слейвов
Записей в таблице users	995 000 (реальные ФИО из CSV)
Инструмент нагрузки	k6 v0.57.0, длительность 30 с
Уровни нагрузки (VU)	1 / 10 / 50 / 100
Профиль чтения	50% /user/get/{id} + 50% /user/search

Роутинг в приложении. Реализован ConnectionRouter (аналог Spring ReplicationRoutingDataSource): запись (CreateAsync) всегда идёт на мастер, чтение (GetByIdAsync, SearchAsync) — round-robin по слейвам. Переключение флагом Replication:UseReplicas в appsettings.json.

2. Результаты чтения: мастер vs слейвы

baseline — все чтения на мастер. **slaves** — чтения распределены по двум слейвам.

GET /user/get/{id} — p95 latency, мс

VUs	baseline (master)	slaves	throughput baseline, rps	throughput slaves, rps
1	5.0	10.8	63.0	25.0

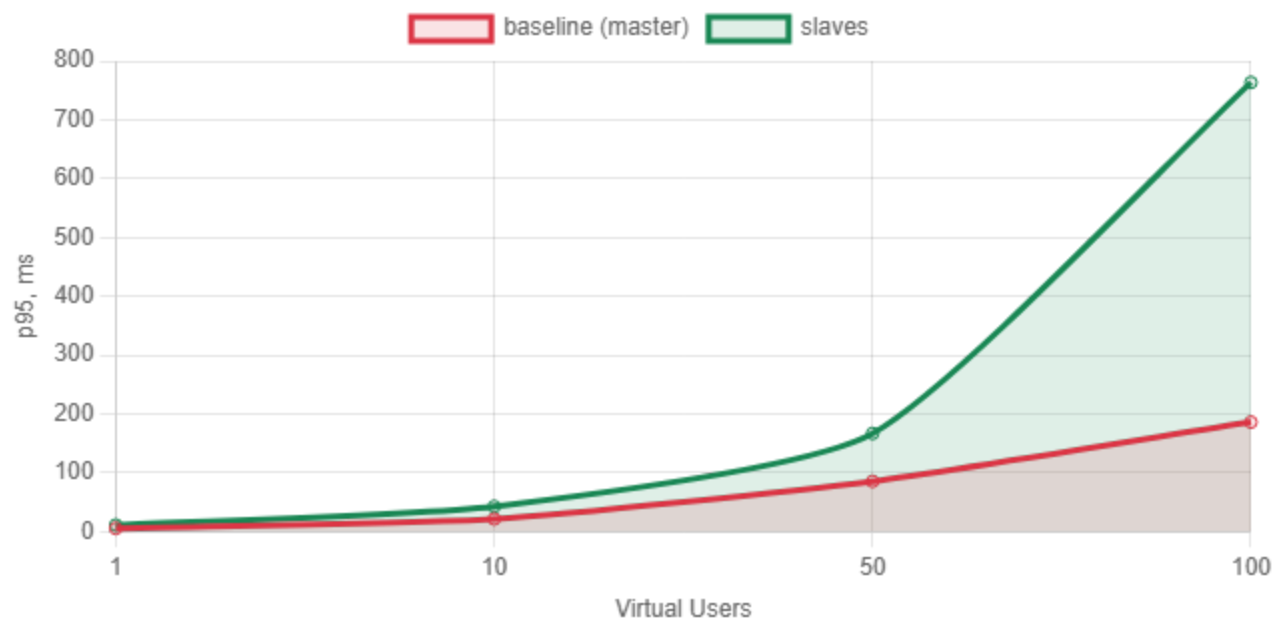
10	21.1	42.4	222.6	169.7
50	85.1	166.5	209.3	183.0
100	185.8	763.4	218.2	127.1

GET /user/search — p95 latency, mc

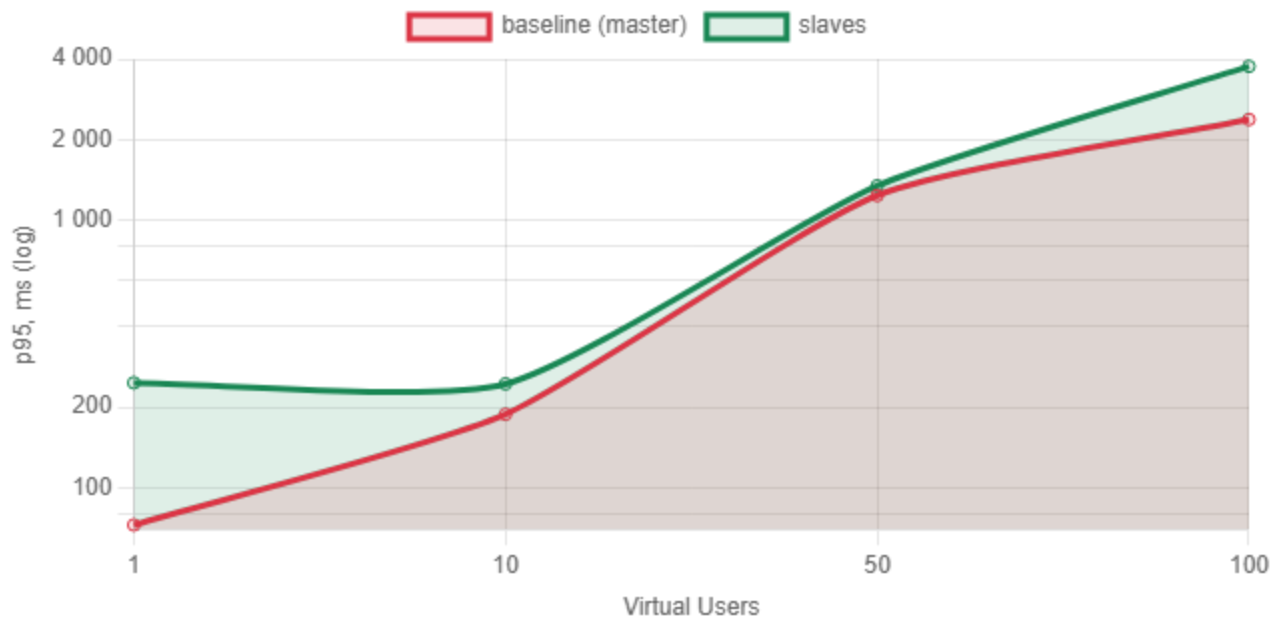
VUs	baseline (master)	slaves
1	72.7	246.9
10	188.4	244.7
50	1236.2	1343.0
100	2376.7	3754.7

3. Графики

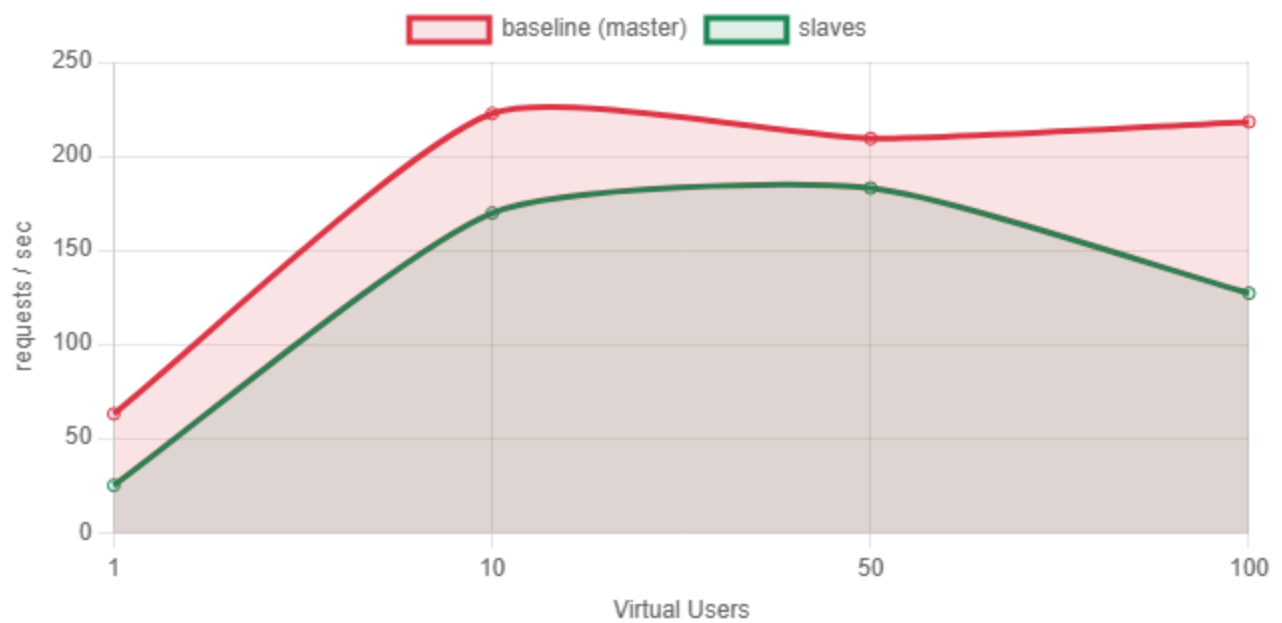
Latency /user/get p95



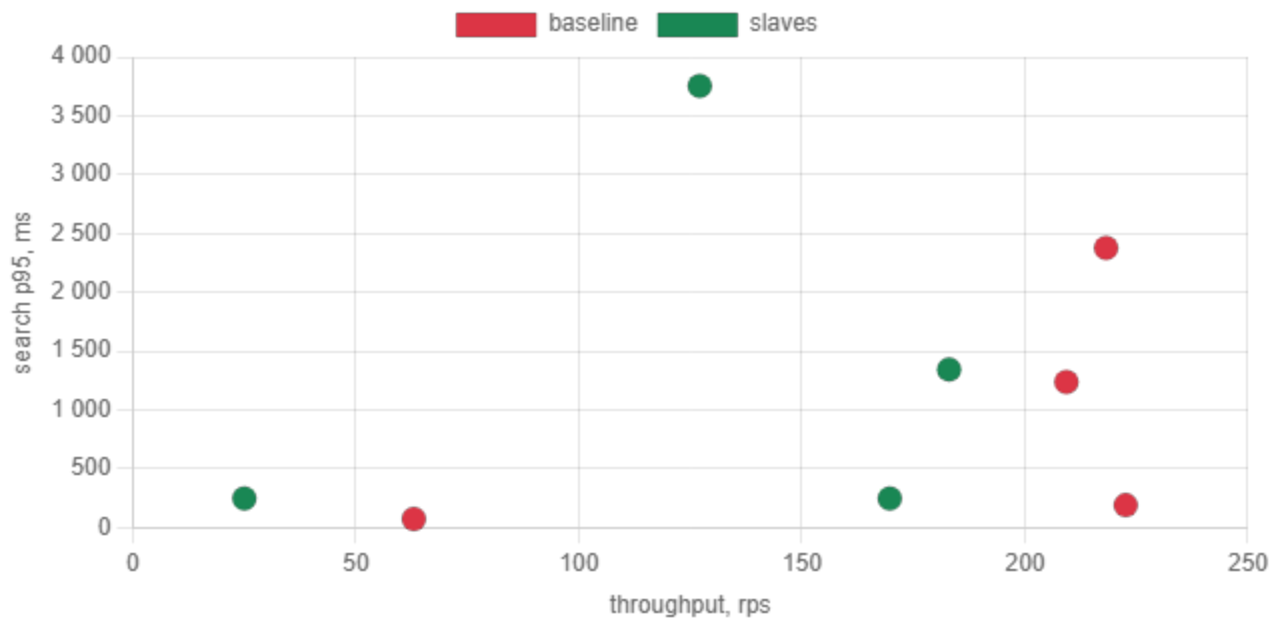
Latency /user/search p95



Throughput (rps)



Throughput vs Latency (search)



4. Интерпретация: почему слейвы не быстрее?

На графиках видно, что распределение чтения по слейвам **не дало прироста** — latency и throughput сопоставимы или чуть хуже baseline. Это **ожидаемый и честный результат для стенда на одной физической машине**, и вот почему:

- **Нет нового железа.** Все три узла MySQL живут в одном Docker/WSL2 на одном CPU и одном диске. Раскидывание чтения по двум слейвам не добавляет мощности — та же машина конкурирует сама с собой, плюс мастер продолжает гнать binlog на оба слейва (dump-потoki), а слейвы крутят IO+SQL apply. Итог — накладные расходы без выигрыша.
- **Round-robin «разбавляет» кэш.** При 1 VU запросы прыгают между двумя слейвами, у каждого buffer pool видит лишь половину запросов → хуже cache-hit, отсюда рост p95 на низкой нагрузке (246 мс против 73 мс).
- **Реальный выигрыш read-replica виден только на отдельном железе:** когда каждая реплика — отдельный сервер, N реплик дают $\approx N \times$ read-capacity. На одном ноутбуке это физически не воспроизвести.

Что репликация реально даёт на этом стенде:

1. **Разгрузку мастера от чтения** — мастер освобождается под запись (нагрузка на запись не конкурирует с тяжёлыми search-запросами).
2. **Отказоустойчивость (HA)** — при смерти мастера слейв промоутируется в мастер (см. раздел 6, потерь транзакций нет).

5. Кворумная (semi-sync) репликация

Мастер подтверждает коммит клиенту только после ACK минимум от одного слейва:

```
-- на мастере
INSTALL PLUGIN rpl_semi_sync_source SONAME 'semisync_source.so';
SET GLOBAL rpl_semi_sync_source_enabled = 1;
SET GLOBAL rpl_semi_sync_source_wait_for_replica_count = 1;    -- кворум = 1 из 2
SET GLOBAL rpl_semi_sync_source_timeout = 10000;

-- на каждом слейве
INSTALL PLUGIN rpl_semi_sync_replica SONAME 'semisync_replica.so';
SET GLOBAL rpl_semi_sync_replica_enabled = 1;
STOP REPLICA IO_THREAD; START REPLICA IO_THREAD;
```

Показатель на мастере	Значение
Rpl_semi_sync_source_status	ON
Rpl_semi_sync_source_clients	2 (оба слейва)

6. Файловер: убийство мастера под нагрузкой на запись

Сценарий: непрерывная запись в таблицу `failover_test` с подсчётом успешных коммитов на клиенте → `docker kill mysql-master` → промоут самого свежего слейва → проверка потерь.

2 432

успешных коммитов (клиент)

2 432

строк на новом мастере

0

потеряно транзакций

15

ошибок после смерти мастера

Потерь нет. Клиент получил подтверждение по 2432 коммитам — ровно столько же оказалось на новом мастере после промута. Это гарантия semi-sync: каждый подтверждённый клиенту commit был реплицирован минимум на один слейв до ответа клиенту, поэтому смерть мастера его не теряет. 15 ошибок — это попытки записи уже *после* kill, клиент их успешными не считал.

Шаги файловера

1. Сравнить @@GLOBAL.gtid_executed обоих слейвов, выбрать с наибольшим номером транзакции (оба применили ...:1-2681 — данные идентичны).
2. На выбранном слейве: STOP REPLICA; RESET REPLICA ALL; SET GLOBAL read_only=0;
3. Второй слейв перенаправить на нового мастера: CHANGE REPLICATION SOURCE TO SOURCE_HOST='mysql-slave1'...
4. Проверка: запись в новый мастер реплицируется на оставшийся слейв. 

7. EXPLAIN поискового запроса (на слейве)

```
EXPLAIN SELECT id FROM users FORCE INDEX(idx_name_search)
WHERE first_name LIKE 'Ив%' AND second_name LIKE 'Ив%' ORDER BY id;
```

id	select_type	table	type	key	key_len	rows	Extra
1	SIMPLE	users	range	idx_name_search	804	1	Using where; Using index

Слейвы получили ту же схему и индекс idx_name_search (first_name, second_name) из Д32 через репликацию DDL — поиск на них работает по индексу (type=range, Using index), как и на мастере.

8. Выводы

- Настроена потоковая GTID-репликация 1 мастер + 2 слейва, чтение разнесено на слейвы, запись — на мастер (application-level routing).
- На одной физической машине read-replica **не ускоряет чтение** (нет нового железа), но разгружает мастер и даёт отказоустойчивость.
- Semi-sync с кворумом 1/2 обеспечил **файловер без потери транзакций**: 2432 подтверждённых коммита пережили kill -9 мастера.